

■ MATERIAL EDUCATIVO

SESIÓN 3

HTML/CSS como interfaz

Fase:	Fase 1 – Pensar antes de programar
Duración:	1 hora
Nivel:	Fundamentos de interfaz
Prerequisitos:	Sesiones 1 y 2 completadas

1. ■ Idea central de la sesión

Al final de esta hora, debe quedar absolutamente claro:

HTML y CSS NO son programación. Son declaración de estructura y apariencia. El código que 'hace cosas' vendrá después (JavaScript). HTML/CSS son la interfaz visible del sistema, no su cerebro.

Analogía correcta (no engañosa):

- HTML = esqueleto de una casa (paredes, puertas, ventanas)
- CSS = pintura, decoración, distribución de muebles
- JavaScript = electricidad, fontanería, sistemas que funcionan

Malentendido a evitar:

"Hacer una web bonita" ≠ "programar una app"

2. ■ Conexión con conocimientos previos

Desde Scratch:

En Scratch, cuando diseñabas un personaje o fondo:

- **Apariencia:** cambiar disfraz, tamaño, posición → similar a **CSS**
- **Estructura:** qué elementos hay en pantalla → similar a **HTML**
- **Lógica:** cuándo hacer al presionar, repetir, si... → esto **NO** está en HTML/CSS

Desde las Sesiones 1 y 2:

Ya sabes que tu app tiene estas partes: Menú principal, cada juego, y sistema de navegación. Hoy vamos a crear **solo la cara visible del menú**, sin que funcione todavía.

Pregunta clave:

"Si el menú se ve pero no hace nada cuando haces clic, ¿está programado?"

Respuesta: **No**. Está *declarado* (existe visualmente), pero no *programado* (no tiene comportamiento).

3. ■ Explicación clara del concepto

¿Qué es HTML?

HyperText Markup Language = lenguaje de marcado (etiquetas).

- **No es:** un lenguaje de programación
- **Sí es:** una forma de describir qué elementos hay en una pantalla

```
<button>Jugar Tic Tac Toe</button>
```

Esto dice: "existe un botón que tiene este texto". **NO** dice: "cuando lo pulses, pasa esto".

¿Qué es CSS?

Cascading Style Sheets = hojas de estilo en cascada.

- **No es:** lógica
- **Sí es:** instrucciones de cómo se ve cada cosa

```
button {  
background-color: blue;  
font-size: 20px;  
}
```

Esto dice: "los botones se ven azules y con letra grande". **NO** dice: "si haces clic, cambia de pantalla".

La diferencia crítica:

Lenguaje	Tipo	Qué hace	Qué NO hace
HTML	Marcado	Declara "hay un botón aquí"	No sabe qué pasa si lo pulsas
CSS	Estilos	Declara "este botón es azul"	No cambia nada por sí solo
JavaScript	Programación	Dice "cuando pulses, haz esto"	NO declara apariencia

¿Por qué importa entenderlo?

Porque cuando uses IA, si le pides "Hazme un menú bonito", te dará **HTML + CSS** (estructura + estilo). Pero si le pides "Haz que al pulsar este botón vaya al juego", necesita **JavaScript** (lógica). **Confundir esto = código desordenado.**

4. ■ Diseño antes de código

Antes de escribir HTML, responde:

Pregunta 1: ¿Qué elementos tiene tu menú?

NO empieces a escribir código todavía. Dibuja o lista:

- ¿Tiene título?
- ¿Cuántos botones?
- ¿Qué dice cada botón?
- ¿Hay algo más? (instrucciones, decoración, logo...)

Ejemplo de lista:

```
Título: "Mini-Juegos"  
Botón 1: "Tic Tac Toe"  
Botón 2: "Ahorcado"  
Botón 3: "Salir" (?)
```

Pregunta 2: ¿Qué responsabilidad tiene cada elemento?

- **Título:** mostrar nombre de la app (no hace nada más)
- **Botón:** ser clicable (pero HOY no hará nada, solo existirá)
- **Fondo:** contener todo

Pregunta 3: ¿Cómo están organizados?

Vertical, horizontal, centrado, esquinas... **Esto es diseño de interfaz.** No es "hacer bonito", es **decidir estructura.**

5. ■ Ejemplo aplicado al proyecto

Escenario:

Vas a crear **solo la pantalla del menú principal**, estática (sin funcionar).

Estructura mínima en HTML:

```
<!DOCTYPE html> <html> <head> <title>Mini-Juegos</title> <link rel="stylesheet" href="styles.css"> </head> <body> <div class="menu"> <h1>Mini-Juegos</h1> <button class="game-button">Tic Tac Toe</button> <button class="game-button">Ahorcado</button> </div> </body> </html>
```

Análisis:

- <div class="menu"> = contenedor (caja que agrupa todo)
- <h1> = título grande
- <button> = botón (sin funcionalidad aún)

Estilos básicos en CSS:

```
body { background-color: #1a1a2e; color: white; font-family: Arial, sans-serif; }  
.menu { text-align: center; padding-top: 100px; } .game-button { display: block;  
margin: 20px auto; padding: 15px 40px; font-size: 18px; background-color: #16213e;  
color: white; border: 2px solid #0f3460; cursor: pointer; } .game-button:hover {  
background-color: #0f3460; }
```

Análisis:

- .menu = reglas para el contenedor
- .game-button = reglas para botones
- :hover = cómo se ve cuando pasas el ratón (feedback visual, pero NO lógica)

Lo que FALTA (y es intencional):

- No pasa nada al hacer clic → eso será JavaScript (próxima sesión)
- No hay navegación → eso será JavaScript
- No hay estado → eso será JavaScript

Hoy solo creas la "cara" del menú.

6. ■ Uso guiado de agentes de IA

■ Qué pedirle a un agente esta semana:

Ejemplo 1 (estructura):

"Dame el HTML de un menú con un título 'Mini-Juegos' y dos botones: 'Tic Tac Toe' y 'Ahorcado'. Solo HTML, sin JavaScript."

Ejemplo 2 (estilos):

"Dame CSS para centrar estos botones verticalmente, con fondo oscuro y botones azules."

Ejemplo 3 (comparación):

"Muéstrame dos formas diferentes de organizar estos elementos con CSS: una centrada verticalmente y otra con los botones a la izquierda."

■ Qué revisar OBLIGATORIAMENTE:

1. Separación de archivos:

- ¿Está el CSS en un archivo .css separado?
- ¿O lo metió todo en el HTML con <style>?

Bueno: <link rel="stylesheet" href="styles.css">

Malo (mezcla todo): <style> button { color: blue; } </style>

2. Nombres de clases:

- ¿Tienen sentido? (menu, game-button)
- ¿O son genéricas? (div1, boton, cosa)

3. ¿Metió JavaScript sin que lo pidieras?

<!-- ■ MALO: no pediste esto -->
<button onclick="startGame()">Jugar</button>

Si aparece onclick, addEventListener, o cualquier .js, **bórralo**. Hoy NO toca lógica.

■■ Errores típicos que puede cometer la IA:

Error 1: Mezclar todo en un archivo

Corrección: Tres archivos separados: index.html, styles.css, (y script.js cuando toque).

Error 2: Dar CSS innecesariamente complejo

Pregunta crítica: "¿Necesito entender todo esto HOY?" Si la respuesta es no, **simplifica**.

Error 3: Usar frameworks sin que lo pidas

Hoy: HTML y CSS **puros**. No frameworks. No librerías.

7. ■ Preguntas de pensamiento crítico

NO preguntas de memoria, sino de razonamiento:

Pregunta 1:

"Si cambias el color de fondo en el CSS, ¿cambia algo en el HTML?"

Reflexión esperada: No. HTML dice "qué hay", CSS dice "cómo se ve". Son independientes.

Pregunta 2:

"Si borras todos los estilos (CSS), ¿desaparecen los botones?"

Reflexión esperada: No. Los botones siguen existiendo (HTML), solo se ven feos (sin CSS).

Pregunta 3:

"¿Puedes hacer que al pulsar un botón cambie de pantalla solo con HTML y CSS?"

Reflexión esperada: No. HTML/CSS no tienen lógica. Para eso necesitas JavaScript.

Pregunta 4:

"Si la IA te da un CSS de 200 líneas para dos botones, ¿es bueno o malo?"

Reflexión esperada: Probablemente malo. CSS largo ≠ CSS bueno. Simplicidad > complejidad innecesaria.

Pregunta 5:

"¿Qué pasa si mañana quieres agregar un tercer botón?"

Reflexión esperada: Si tu CSS está bien hecho: solo añades un `<button>` en HTML. Si está mal: tienes que tocar CSS también. Esto es diseño escalable.

8. ■ Mini-reto para 1 hora

■ Objetivo:

Crear la pantalla del menú principal (estática) con decisiones conscientes.

Pasos (con decisiones, no receta):

Paso 1: Diseñar en papel (10 min)

- Dibuja: dónde va el título, cuántos botones (2 juegos mínimo), disposición (vertical, horizontal, centrado...)
- **Decisión:** ¿Qué transmite tu diseño? (diversión, minimalismo, retro...)

Paso 2: Escribir HTML básico (15 min)

- Crea index.html con: estructura <html>, <head>, <body>, contenedor del menú, título, botones (uno por juego)
- **Decisión:** ¿Usas <div> o <section>? ¿Por qué?

Paso 3: Escribir CSS limpio (20 min)

- Crea styles.css con: colores de fondo, estilo de botones, centrado
- **Decisión:** ¿Colores brillantes o sobrios? ¿Por qué?

Paso 4: Revisar (10 min)

- ¿Se parece a tu diseño en papel?
- ¿Puedes añadir un tercer botón fácilmente?
- ¿El CSS es reutilizable?

Paso 5: Experimento final (5 min)

- Borra **todo** el CSS.
- **Pregunta:** "¿Sigue funcionando? ¿Por qué?"
- **Respuesta esperada:** Sí, porque HTML es la estructura. CSS solo embellece.

■ Criterio de éxito:

- Menú visible con título y botones
- CSS separado del HTML

■ Puedes explicar qué hace cada línea de CSS

■ Si agregas un botón, no rompe el diseño

■ **NO es éxito si:**

- Los botones funcionan (hoy no deben)
- Hay JavaScript mezclado
- Copiaste código sin entenderlo

9. ■ Qué NO hacer

■ Error 1: Empezar a escribir código sin diseñar

Síntoma: Síntoma: "Voy probando a ver qué sale"

Consecuencia: Consecuencia: Código desorganizado que hay que rehacer.

Solución: Solución: Siempre: **papel** → **diseño** → **código**.

■ Error 2: Mezclar HTML, CSS y JavaScript en un archivo

Síntoma: Síntoma: `<button style="color: blue;" onclick="alert('hola')">`

Consecuencia: Consecuencia: Imposible de mantener. Caos total.

Solución: Solución: HTML: estructura (index.html), CSS: estilos (styles.css), JavaScript: lógica (script.js, próxima semana)

■ Error 3: Pedir a la IA "hazme un menú bonito"

Síntoma: Problema: No especificas qué quieres. Te dará algo genérico.

Consecuencia: Mejor: "Dame HTML de un menú con título 'X' y N botones, organizados verticalmente."

Solución:

■ Error 4: Usar CSS que no entiendes

Síntoma: Síntoma: `.button { transform: matrix(1, 0, 0, 1, 0, 0); }`

Consecuencia: Si no sabes qué hace: **bórralo**.

Solución: Principio: Solo código que puedes explicar.

■ Error 5 (de la IA): Dar frameworks sin avisar

Síntoma: Ejemplo: `<link href="https://cdn.tailwindcss.com" rel="stylesheet">`

Consecuencia: Problema: Tailwind, Bootstrap, etc. son herramientas avanzadas. Hoy: **CSS puro**.

Solución: Si la IA lo hace: "Rehaz esto sin frameworks, solo CSS básico."

■ Resumen de la sesión (cheat sheet)

Concepto	Definición corta	Ejemplo
HTML	Estructura (qué hay)	<code>&lt;button>Jugar&lt;/button></code>
CSS	Apariencia (cómo se ve)	<code>button { color: blue; }</code>
JavaScript	Lógica (qué hace)	<code>button.onclick = ...</code> ← próxima semana
Separación	Cada cosa en su archivo	<code>.html / .css / .js</code>
Declarativo	Dices "qué", no "cómo"	HTML/CSS no "ejecutan" nada

■ Entregable de esta sesión

Al final de la hora, debes tener:

1. ■ Dibujo del menú (papel o digital)
2. ■ Archivo `index.html` con estructura del menú
3. ■ Archivo `styles.css` con estilos básicos
4. ■ Respuesta escrita: "¿Qué diferencia hay entre HTML y JavaScript?"

Respuesta esperada (en tus palabras):

HTML dice qué elementos hay (botones, títulos...). JavaScript dice qué pasa cuando interactúas con ellos. HTML es estático, JavaScript es dinámico.

■ Próxima sesión (adelanto)

Tema: Introducción a JavaScript como "cerebro"

Qué haremos: Hacer que uno de esos botones **haga algo** cuando lo pulses.

Conexión: Hoy creaste la cara. La próxima semana, le das inteligencia.

"HTML es el esqueleto. CSS es la piel. JavaScript es el cerebro." Hoy construimos esqueleto y piel. La próxima, el cerebro.

■ Éxito = entender la separación de responsabilidades entre lenguajes, no hacer un menú "bonito".